

1. prompt

- 我将进行《自然语言处理》这门课的 A8 作业：Week 9 随堂 Vibe 实验：机器翻译机制与质量评测系统。本次实验的背景与目的是：在本周的理论课中，我学习了机器翻译（Machine Translation, MT）的发展历程。从早期的“基于规则的机器翻译”（词对词直译、句法转换），到统计机器翻译，再到目前主流的神经机器翻译（NMT）。本次实验，请利用 AI 辅助编程工具（Vibe Coding 模式），开发一个“机器翻译机制对比与评测平台”。你将通过构建直译引擎与神经网络引擎的对比，以及实现机器翻译的自动化评测指标（BLEU），深入理解机器翻译底层的架构差异与评价准则。本次实验分为三个模块，我将分模块发送（请了解这个作业的实验任务总体要求，不需要回复我任何信息）。实验任务为：构建“机器翻译对比与评测Web系统”。总体要求为：使用 Python 的 Streamlit 框架，结合 Hugging Face 的 transformers 库和 nltk 库，构建包含三种核心翻译体验的交互式应用。

已了解整体任务要求与背景。我会按你后续分模块的内容逐步实现这个 Streamlit 机器翻译对比与评测系统。

- 模块 1: 神经机器翻译引擎 (NMT Engine)
 1. 对应课件: 机器翻译原理与神经机器翻译架构
 2. 具体要求: 在 Streamlit 应用的第一个标签页, 实现一个神经机器翻译模块。提供一个文本输入框, 允许用户输入英文句子。使用 Hugging Face transformers 库的 pipeline 功能, 加载轻量级的英译中模型 (如 Helsinki-NLP/opus-mt-en-zh)。输出模型生成的中文译文, 并要求界面具有加载中的交互提示 (Spinner)。

模块 1 已完成并可直接运行，包含你要求的全部点：

1. 第一个标签页实现了 NMT 英译中功能。
2. 提供英文输入框。
3. 使用 `transformers.pipeline` 加载 `Helsinki-NLP/opus-mt-en-zh`。
4. 输出中文译文。
5. 翻译过程带 `st.spinner` 加载提示。

- [app.py]
- [requirements.txt]

 $\frac{1}{4}$

```
pip install -r requirements.txt
streamlit run app.py
```

后续你把模块 2 发我，我会直接接在这个应用里继续实现。

- 模块 2：基于规则的直译 vs. 神经网络意译 (Rule-based vs NMT)
 1. 对应内容：基于规则的机器翻译（词典直接替换）及其局限性
 2. 具体要求：在第二个标签页，实现一个对比模块。编写一个 Python 函数，模拟早期的『基于规则的机器翻译』：利用一个预定义的英汉词典字典 (Dictionary)，对输入的英文句子进行基于空格分词的『逐词直译』（如果词典中没有的词直接保留英文）。将模块 1 中的 NMT 译文与这个『逐词直译』的结果在页面上并排展示，进行效果对比。

GPT-5.3-Codex-Medium:

模块 2 已完成，已接入现有 Streamlit 应用并可运行。

已实现内容：

1. 新增规则直译函数 `rule_based_translate` :
 - 使用预定义英汉词典 `RULE_DICT`
 - 按空格分词逐词翻译
 - 词典中没有的词保留英文
 - 尽量保留词尾标点（如 `dogs.`）
2. 第二个标签页已实现对比模块：
 - 输入英文句子
 - 同时生成「规则直译」和「NMT 译文」
 - 两种结果在页面左右并排展示
3. 复用模块 1 的 NMT 模型：
 - 通过 `nmt_translate` 统一调用 `Helsinki-NLP/opus-mt-en-zh`

已更新文件：

- [app.py]

如果你现在发模块 3，我可以继续直接在这个版本上完成 BLEU 自动评测页。

- 模块 3：机器翻译质量自动评测 (BLEU Score)
 1. 对应内容：机器翻译评测方法
 2. 具体要求：在第三个标签页，实现机器翻译自动评测功能。页面需要三个输入区域：1. 待翻译英文原文；2. 标准的中文参考译文 (Reference)；3. 机器生成的候选译文 (Candidate，可直接调用模块1生成)。使用 `nltk.translate.bleu_score` 模块，计算机器译文与参考译文之间的 BLEU 分数。在页面上输出最终的 BLEU 得分，并解释这个分数代表的意义。

GPT-5.3-Codex-Medium:

模块 3 已完成，并已经整合进第三个标签页。

已实现功能：

1. 三个输入区域

- 英文原文
- 中文参考译文 (Reference)
- 中文候选译文 (Candidate)

2. 候选译文自动生成

- 提供“调用 NMT 生成候选译文”按钮
- 直接复用模块 1 的 `Helsinki-NLP/opus-mt-en-zh` 结果填入 Candidate

3. BLEU 自动计算

- 使用 `nltk.translate.bleu_score` 的 `sentence_bleu`
- 对中文做字符级 token (适合中文无空格文本)
- 使用平滑函数 `SmoothingFunction().method1` 避免短句 BLEU 过度归零

4. 结果展示与解释

- 显示 BLEU 数值 (0~1)
- 页面说明“分数越高，候选译文与参考译文越接近”
- 按分数区间给出文字化解读

已更新文件：

- [app.py]

你现在的三个模块已经是一个完整可演示版本了。

2. 实验总结

本次实验围绕“翻译机制对比与质量评测”完成了一个可交互的 Streamlit 系统，我对课堂中的机器翻译理论有了更直观的理解。通过神经机器翻译模块，我看到 NMT 能在多数场景下结合上下文完成较自然的意译；而在规则直译模块中，逐词替换虽然实现简单、可解释性强，但在长句结构重组和一词多义判断上明显受限。两者并排对比后，模型能力差异非常清晰。最后在 BLEU 评测模块中，我通过控制参考译文的语序与同义替换，观察到分数对 n-gram 匹配的敏感性，也认识到 BLEU 在“可量化比较”上的价值与“语义覆盖不足”的局限。整体而言，本次实验把“翻译生成”与“自动评价”串成了完整流程，帮助我从工程实现层面真正理解了机器翻译系统的核心问题。

2.1 模块 1 观察任务：输入包含英语俚语或复杂长句的文本（如 "It rains cats and dogs."）。观察神经机器翻译能否正确处理多义词和上下文语境。

- 在观察任务中，我输入了 "It rains cats and dogs." 及多条包含从句的长句。结果显示，神经机器翻译通常不会按字面直译“猫和狗”，而能结合上下文给出“下大雨”等更

自然表达；对复杂长句也能保持主干语义完整。但当句子嵌套层次过深或含罕见俚语时，仍会出现细节偏差，说明其语境建模能力较强，但并非完全稳定。

2.2 模块 2 观察任务：对比两种翻译结果。结合课件思考：基于规则的翻译在处理语序倒装（如定语从句）、一词多义时会遇到怎样不可逾越的鸿沟？

- 对比可见，基于规则的逐词替换只能完成“词面映射”，难以重建句法关系。遇到定语从句或倒装结构时，它往往按原顺序生硬拼接，导致语义主次错位；面对一词多义，也缺乏上下文判别能力，常选错义项。这种方法的瓶颈在于没有真正的语法建模与语境理解，因此很难跨越“可读”到“达意”的门槛。

2.3 模块 3 观察任务：针对同一句英文，分别故意提供一个“词汇完全一样但语序全错”的中文参考，和一个“语序正确但同义词替换”的中文参考。观察 BLEU 分数的变化，体会 BLEU 算法（基于 n-gram 匹配）在评测翻译质量时的优缺点。

- 实验中我发现，若参考译文与候选译文词汇几乎一致但语序被打乱，BLEU 会因高阶 n-gram 断裂而明显下降；而语序基本正确、仅做同义词替换时，分数也会下滑，因为字面重合减少。由此可见，BLEU 优点是可量化、实现简洁，对局部结构敏感；不足是过度依赖表层匹配，难充分反映语义等价与表达自然度。